

GeoAnnotate 项目结构详细说明

2026 年 3 月 20 日

1 项目当前整体架构

当前项目不是传统意义上的纯前后端分仓，而是一个单仓库混合结构：

- 浏览器端页面与交互使用 Next.js
- 服务端逻辑主要使用 CloudBase 云函数
- 数据结构和数据库模式由 Prisma 管理

从职责上看，可以拆成四层：

1. 前端层
2. 后端层
3. 前后端连接层
4. 共享配置层

2 前端层

前端层主要负责：

- 页面展示
- 用户交互
- 标注框绘制
- 地图展示
- 登录与注册
- 管理后台页面
- Wiki 页面壳与导航

2.1 主要目录

- app/
- components/
- app/globals.css
- tailwind.config.ts
- next.config.js

2.2 页面路由结构

根页面

- app/page.tsx: 项目落地页，用于区分 /wiki 和 /app

核心应用页面

- app/(main)/: 核心平台页面的主要实现位置

认证页面

- app/(auth)/: 登录、注册、找回密码

管理后台

- app/admin/: 管理端页面

新路由分区

- app/app/
- app/auth/

这两个目录更多是新的 URL 入口层，很多文件只是转发到原有页面实现。
因此后续真正要改页面内容时，优先修改：

- app/(main)/...
- app/(auth)/...

2.3 前端组件目录

- components/layout/: 导航栏、整体页面壳、路由壳
- components/annotation/: 标注相关组件
- components/map/: 地图显示与选择
- components/battle/: 计时器、计分板
- components/wiki/: Wiki 导航
- components/ui/: 通用 UI 组件

2.4 前端常见修改位置

改首页

- `app/page.tsx`
- `app/(main)/home/page.tsx`

改标注相关界面

- `app/(main)/annotate/mode/page.tsx`
- `app/(main)/annotate/page.tsx`
- `components/annotation/BBoxCanvas.tsx`
- `components/annotation/ThoughtInput.tsx`
- `components/map/LocationMap.tsx`

改 AI 对战界面

- `app/(main)/battle/page.tsx`
- `app/(main)/battle/[sessionId]/play/page.tsx`
- `app/(main)/battle/[sessionId]/result/page.tsx`
- `components/battle/CountdownTimer.tsx`
- `components/battle/BattleScoreBoard.tsx`

改整体布局

- `components/layout/Navbar.tsx`
- `components/layout/AppShell.tsx`
- `components/layout/MainRouteShell.tsx`
- `app/admin/layout.tsx`
- `app/(main)/wiki/layout.tsx`

3 后端层

当前项目的后端不是独立 NestJS / FastAPI 服务，而是以 CloudBase 云函数作为主要后端执行层。

3.1 主要目录

- `cloudfunctions/`
- `prisma/`

3.2 后端负责的内容

- 任务分发
- 标注提交
- 用户信息读取
- AI 对战创建
- AI 对战结果读取
- 奖励与积分逻辑
- 排行榜与历史数据读取
- 管理后台数据处理

3.3 关键云函数

- `cloudfunctions/get-next-task/`
- `cloudfunctions/submit-annotation/`
- `cloudfunctions/create-battle/`
- `cloudfunctions/submit-battle-round/`
- `cloudfunctions/get-battle-result/`
- `cloudfunctions/get-user-profile/`
- `cloudfunctions/get-user-history/`
- `cloudfunctions/get-points-history/`
- `cloudfunctions/get-leaderboard/`
- `cloudfunctions/list-images/`
- `cloudfunctions/review-annotation/`
- `cloudfunctions/export-annotations/`

3.4 数据库层

数据库结构主要在：

- `prisma/schema.prisma`

如果要改数据库结构，优先修改这个文件，并同步检查对应云函数和前端调用层。

4 前后端连接层

前端和后端之间并不是页面直接访问数据库，而是通过统一的调用层转发到云函数。

最关键的文件是：

- `lib/cloudbase.ts`

它的作用包括：

- 封装 CloudBase SDK 初始化
- 封装登录、注册、找回密码
- 封装云函数调用
- 给前端页面提供统一的方法名

如果后端新增或修改了接口，通常需要同步修改：

- `cloudfunctions/...`
- `lib/cloudbase.ts`

5 配置与共享层

这部分不直接属于前端界面，也不属于后端业务逻辑，但它决定了项目的整体骨架。

5.1 主要目录和文件

- `features/`
- `types/`
- `lib/modes.ts`
- `package.json`
- `tsconfig.json`
- `next.config.js`

5.2 各部分作用

- `features/annotation/`: 标注模式与标注类型配置
- `features/battle/`: 对战模式、AI 对手、回合数、时间配置
- `features/admin/`: 管理后台导航配置
- `features/platform/`: 平台入口与路由结构说明
- `types/`: 共享类型定义
- `lib/modes.ts`: 统一导出模式配置

6 如何快速判断改哪里

目标	优先修改位置	说明
改页面样式和布局	<code>app/(main)</code> 、 <code>components</code>	前端页面与组件
改导航和壳层	<code>components/layout</code> 、 <code>app/admin/layout.tsx</code>	页面骨架与导航结构
改接口调用方式	<code>lib/cloudbase.ts</code>	前后端调用桥梁
改业务逻辑	<code>cloudfunctions</code>	云函数处理数据库和逻辑
改数据库结构	<code>prisma/schema.prisma</code>	数据表与字段定义
改模式与配置	<code>features</code> 、 <code>lib/modes.ts</code>	全局配置与共享常量

7 一句话总结

当前项目可以简单理解为：

- `app/` + `components/` = 前端
- `cloudfunctions/` + `prisma/` = 后端
- `lib/cloudbase.ts` = 前后端连接层
- `features/` + `types/` + 配置文件 = 项目框架与共享配置

如果后续要同步到 GitHub README，建议用精简版放在 README 中，把详细版长期维护在 docs/ 目录。